



Python Project

Digital Board Game Development with Python

Presented by

6788024 Saran Intornphu

6788038 Panlapas Keerin

**A Report Submitted in Partial Fulfillment of
the Requirements for**

ITCS251 Programming in Python

**Faculty of Information and Communication Technology
Mahidol University
Academic year 2/2025**

Project overview

This project presents the development of a digital version of the board game *Catan* using Python. The objective of the project is to simulate the essential mechanics of the original board game in a digital environment, allowing players to interact through a graphical user interface (GUI). The system supports multiple players and replicates key gameplay elements such as resource generation, construction of structures, turn-based actions, and victory conditions.

The application is designed using a modular architecture, where each component is responsible for a specific functionality. This separation improves readability, maintainability, and scalability of the system, making it easier to expand or modify in the future.

Project structure

```
catan_game/      # Main project directory
├── main.py       # Entry point of the application
├── requirements.txt # Project dependencies (standard library only)
├── README.md     # Project documentation and usage guide
├── game/         # Core game logic
│   ├── board.py  # Game board representation
│   ├── player.py # Player data and state management
│   ├── game_engine.py # Main game controller
│   └── dice.py   # Dice rolling logic
├── gui/          # User interface components
│   ├── main_window.py # Main application window
│   ├── board_view.py  # Visual representation of the board
│   └── controls.py    # User interaction controls
├── data/         # Data management
│   ├── database.py   # Database initialization and connection
│   └── models.py     # Data models
├── utils/        # Helper functions and constants
│   ├── constants.py  # Game constants and configuration
│   └── helpers.py    # Utility functions
└── assets/       # Static resources
    ├── images/      # Game images and textures
    └── sounds/       # Audio files
```

System Architecture

The overall system is divided into four major components, each playing a distinct role in the application:

1. GUI Layer (gui /)

The GUI layer is responsible for managing all visual and user interaction aspects of the game. It serves as the bridge between the player and the internal game logic. Through this layer, users can view the game board, interact with controls, and perform in-game actions such as building or ending their turn.

- Main window display
- Game board rendering
- Player controls and actions

2. Game Logic Layer (game /)

This layer contains the core functionality of the game and acts as the brain of the application. It processes all game rules and ensures that every action performed by the player follows the defined mechanics of the game.

The **GameEngine** plays a central role by coordinating all components and managing the flow of the game. It communicates with the board, players, and dice system to maintain the current state of the game.

- GameEngine (main controller)
- Board (map and tiles)
- Player (player data and resources)
- Dice (random number generator)

3. Data Layer (data /)

The data layer is responsible for handling persistent storage within the system. It ensures that important game information, such as game history, can be saved and retrieved when necessary. This layer also initializes the database at the start of the program.

- Database initialization
- Game history storage

4. Utility Layer (utils/)

This layer provides supporting functionality used throughout the system. It includes constants such as building costs, validation functions to ensure correct gameplay actions, and helper utilities to simplify repeated operations.

- Constants (e.g., building costs)
- Validation functions
- Helper methods

Game Workflow

1. Program Initialization

The execution of the program begins in the `main.py` file, which acts as the entry point of the application. When the program starts, the system first initializes the database by calling the `init_db()` function. This step ensures that all necessary tables and data structures are ready for storing game-related information.

After the database setup is complete, the graphical user interface is launched by creating an instance of the `MainWindow` class. At this stage, the application becomes interactive, allowing users to start a new game and input player information.

2. Game Setup Phase

Once the user starts a new game, the system initializes the **GameEngine** with the provided list of player names. The engine performs validation to ensure that the number of players falls within the allowed range of 2 to 4 players. This constraint maintains the balance and rules of the game.

After validation, the system proceeds to initialize key components, including the game board, player objects, and the dice mechanism. Each player is assigned initial properties such as resources and structures.

The game then transitions into its first operational state:

- `SETUP_SETTLEMENT_1`

3. Setup Gameplay

Phase 1: First Placement

During this phase, each player takes turns placing their first settlement and a connected road on the board. This step is critical because it determines the initial resource opportunities for each player.

Phase 2: Second Placement

In the second setup phase, players repeat the placement process. However, the order of turns may be reversed, giving the last player from the previous round an advantage by placing first in this phase. This mechanic helps balance gameplay.

4. Main Gameplay Loop

After completing the setup phases, the game enters its main loop, where players take turns performing actions until a winner is determined.

4.1 Dice Roll Phase (ROLL_DICE)

At the beginning of each turn, the current player rolls the dice using the system's dice module. The outcome of the dice roll determines how resources are distributed among players.

If the result is not equal to 7, the system allocates resources to players based on the tiles connected to their settlements or cities. Each tile corresponds to a specific resource type and number.

If the result is 7, the normal resource distribution is skipped, and the game transitions into a special phase known as the robber phase.

4.2 Robber Phase (ROBBER)

The robber phase introduces an interactive and strategic element to the game. The current player is required to move the robber to a new tile on the board. Once placed, the robber blocks resource generation from that tile.

Additionally, the player may choose another player who has a settlement or city adjacent to the selected tile and steal one resource from them. This introduces both competition and tactical decision-making.

4.3 Action Phase (ACTIONS)

During the action phase, the active player has the freedom to perform multiple actions in any order, as long as they have sufficient resources and the actions comply with game rules.

Available Actions:

- Build:
 - Roads

- Settlements
- Cities
- Trade with bank (default ratio 4:1)
- End turn

Each action is validated by the system using predefined rules and cost structures to ensure fairness and consistency.

4.4 Turn Transition

After completing their actions, the player ends their turn. The system updates the current player index to pass control to the next player in sequence.

- `current_player_index`
- Phase resets to `ROLL_DICE`

This cycle continues repeatedly, forming the core gameplay loop.

4.5 Game End (GAME_OVER)

The system continuously monitors each player's victory points throughout the game. When a player reaches the required number of victory points, the game immediately transitions to the game over phase.

At this point:

- The game stops
- A winner is declared

Game Engine Responsibilities

The **GameEngine** serves as the central controller of the entire application. It coordinates all components and ensures that the game progresses smoothly according to predefined rules.

Instead of handling isolated tasks, the GameEngine continuously processes player input, updates the game state, and communicates with other modules such as the board and player objects. It is responsible for maintaining consistency across all aspects of the game.

Main Responsibilities

- Manage turn order
- Control game phases
- Handle dice and resource distribution
- Validate and execute player actions
- Handle robber mechanics
- Check win conditions

System Interaction Flow

- GUI → GameEngine → Board/Player → Result → GUI Update

How to Run the Program

1. Requirements

Before running the program, the user must ensure that the required environment is properly set up. The project requires Python version 3.10 or higher. Additionally, any necessary libraries used for the graphical interface must be installed.

- Python 3.10 or higher
- Required libraries (e.g., Tkinter or PyQt)

2. Project Setup

To begin using the program, the user must first extract the project files and open the folder in a code editor such as Visual Studio Code. This allows easy navigation and execution of the program files.

3. Run the Application

The program can be executed through the terminal using the following command:

- `python main.py`

4. Program Execution Flow

When the program runs, the system first initializes the database and prepares all necessary components. The main window then appears, allowing the user to interact with the application.

The user begins by starting a new game and entering player names. The system then transitions into the setup phase, followed by the main gameplay loop. All actions are performed through the GUI, providing an interactive experience.

5. Example Gameplay Flow

- Player 1 places settlement and road
- Player 2, 3, 4 repeat
- Dice is rolled
- Resources are distributed
- Player builds or trades

- Turn ends → next player
- Repeat until a player wins

Conclusion

This project successfully demonstrates the implementation of a digital board game using Python, emphasizing structured design and modular development. By separating the system into clear components and maintaining a well-defined game workflow, the application achieves both functionality and maintainability.

The project also highlights the use of object-oriented programming principles in managing complex interactions between players, game states, and system components.